



Universidade de Brasília
Departamento de Ciência da Computação
Sistemas de Informação 2026.1

Relatório Final

222005395 Lurian Correia Lima

222001369 Nicolas Meloni

232002520 Giovanni Daldegan

242001491 Gabriel de Medeiros Matos

28 de junho de 2026

1. Introdução

Hoje em dia, torna-se cada vez mais nítida a necessidade de uma movimentação rápida e eficiente de recursos médicos em hospitais, tanto para agilizar o atendimento aos pacientes quanto para facilitar o trabalho dos profissionais de saúde. Para atender a essa exigência e modernizar o cotidiano hospitalar, foi idealizado um sistema de controle de robôs autônomos dedicado à logística de coleta e entrega de recursos.

O objetivo do projeto é gerenciar uma frota capaz de transportar equipamentos – desde materiais cirúrgicos estéreis até roupas de cama limpas – dos estoques até os agentes solicitantes em locais especificados. Para garantir a eficiência da operação, o sistema central deve ser capaz de receber requisições de forma remota, calcular rotas com base no tempo de espera e gerenciar múltiplas tarefas paralelas para diferentes robôs.

Além do roteamento padrão, por ser um contexto hospitalar, o sistema deve possuir um alto grau de confiabilidade. O sistema precisa monitorar e lidar com imprevistos, como o esgotamento de bateria das unidades durante as fases de coleta e entrega, coordenando o retorno a estações de recarga, o uso de pontos de controle para transferência de carga e a reatribuição de missões em tempo real, alertando os gerentes do setor em caso de falhas críticas. A modelagem precisa desse ecossistema é o primeiro passo para garantir a inovação e a segurança no ambiente hospitalar.

2. Fundamentação Teórica

2.1. Diagrama de Contexto

Para a modelagem do escopo e das fronteiras do sistema, este projeto adotou a abordagem de Diagramas de Contexto, fortemente influenciada pela metodologia Problem Frames. A abordagem busca representar "o Mundo como um conjunto de atores conectados através de interfaces".

O objetivo principal deste modelo não é arquitetar o software internamente, mas sim visualizar o ecossistema no qual a máquina irá operar. Isso permite definir claramente os limites da investigação de requisitos, engajando os *stakeholders* com uma representação fiel do mundo real e de seus processos (o "Mundo-como-ele-é").

A construção do diagrama, a partir do que foi dito, rege-se por regras conceituais claras, destacando-se os seguintes elementos e regras de modelagem:

- **A Máquina:** É o sistema de software que está sendo projetado. É representado como uma "caixa preta" para ocultar intencionalmente a arquitetura interna. Componentes como bancos de dados, painéis e interfaces de usuário não são desenhados como atores, pois pertencem aos processos internos da própria máquina.
- **O Limite do Work:** Representado por uma delimitação tracejada que engloba o escopo do problema. Atores e interações que ocorrem dentro desta linha fazem parte da operação diária que a máquina visa apoiar e otimizar.
- **Atores (Entidades Ativas):** Um ator é estritamente uma entidade capaz de observar e controlar fenômenos. Podem ser indivíduos (pacientes, enfermeiros, gerentes), dispositivos de hardware (robôs, maquinários, sensores) ou sistemas complementares (softwares externos). Entidades passivas, como salas, corredores ou prateleiras vazias, não são consideradas atores.
- **Interfaces (Fenômenos Compartilhados):** As conexões no diagrama não representam fluxos de controle ou verbos de ação vagos, mas sim interfaces: locais no mundo onde dois atores interagem por meio de fenômenos compartilhados. Os rótulos nestas conexões são descritivos e focados em estados ou dados que trafegam.

- **Interações no Mundo Físico:** Uma característica importante desta modelagem é a permissão para mapear interfaces (ou seja, conexões) diretamente entre atores externos à máquina principal. Isso garante a representação de fluxos operacionais que ocorrem no mundo físico (como a transferência de um recurso do robô para o agente solicitante), estabelecendo o contexto no qual o software deve atuar.

A adoção deste modelo permitiu que os limites do sistema de gerenciamento de robôs fossem estabelecidos com alta coesão, separando de forma clara as responsabilidades computacionais da máquina das regras logísticas do contexto hospitalar, sem precisar detalhar tecnologias internas do sistema.

2.2. Identificação de *Stakeholders* e o *Onion Diagram*

Na engenharia de requisitos, um *stakeholder* (parte interessada) é definido como uma pessoa ou um grupo de pessoas que possui interesse ou que é afetado pelo sistema a ser desenvolvido (a Máquina). É um erro comum limitar o escopo de investigação apenas aos clientes e aos usuários diretos, pois frequentemente existem muito mais partes interessadas envolvidas nesse tipo de ecossistema. A falha em considerar *stakeholders* importantes pode causar impactos adversos às pessoas ignoradas, acarretar mudanças custosas durante o ciclo de desenvolvimento ou até mesmo resultar em falha parcial ou total do projeto.

Para evitar essas omissões e garantir uma descoberta sistemática de todos os envolvidos, foi adotado o *Stakeholder Onion Diagram* (Modelo de Cebola) como um checklist de identificação. Este modelo organiza as partes interessadas em camadas que se expandem a partir do produto idealizado:

- **A Área Operacional:** Engloba todos os *stakeholders* que interagem diretamente com o produto. Esta camada inclui os usuários (operadores regulares do sistema), a equipe de suporte operacional (que auxilia os usuários) e as tecnologias de interface, que representam sistemas externos com os quais o produto troca informações.
- **O Negócio Contenedor:** Compreende todos os *stakeholders* que se beneficiam do produto de alguma maneira. Fazem parte deste grupo o cliente (quem financia o desenvolvimento do produto), os consumidores e os beneficiários funcionais, definidos como as pessoas que tiram proveito do trabalho realizado pelos atores da área operacional.
- **O Ambiente Mais Amplo:** Agrupa atores mais distantes, mas que possuem alguma influência ou interesse no produto. Esta camada é composta por órgãos reguladores e inspetores, beneficiários políticos e financeiros, especialistas de domínio, atores afetados negativamente pelo sistema e entidades hostis, como criminosos e atacantes que possam representar riscos à segurança do sistema.
- **A Equipe de Desenvolvimento:** Este grupo abrange as pessoas responsáveis por projetar, desenvolver, testar, operar, manter e evoluir o produto. O modelo defende que o envolvimento precoce de desenvolvedores e testadores na fase de requisitos gera um entendimento compartilhado dos objetivos e melhora a qualidade das especificações levantadas.

A aplicação metodológica deste modelo serve como uma base necessária para levantar os objetivos e as preocupações específicas de cada grupo identificado, garantindo que as restrições e necessidades de todas as esferas sejam contempladas no projeto.

2.3. *Domain Scenarios*

A modelagem dos Domain Scenarios tem como principal vantagem facilitar a comunicação e o alinhamento com os stakeholders. Uma característica fundamental desta metodologia é que os cenários possuem uma estrutura estritamente linear: não contêm condicionais, desvios (branching) ou ciclos (loops). Essa simplicidade é o que os torna tão eficazes para a comunicação. Para a sua

representação prática, a metodologia apoia-se em três notações principais: narrativas textuais organizadas por passos numerados.

Os elementos básicos que compõem a estrutura de um cenário de domínio incluem:

- **Ator Primário:** A entidade, papel ou sistema que inicia e executa a ação;
- **Ação:** O evento em si, sempre expresso por um verbo na voz ativa;
- **Objetos de Domínio:** Os elementos físicos ou digitais que são manipulados ou observados durante a ação;
- **Atores Secundários:** Participantes adicionais que interagem ou testemunham o evento;
- **Contexto e Resultados:** O cenário é delimitado por um título descritivo, um contexto inicial (precondição) e um resultado final (pós-condição), além de anotações opcionais para detalhar questões pendentes.

2.4. Goal Model

O diagrama segue a notação KAOS: os paralelogramos cinza são goals do tipo Achieve (objetivos a serem alcançados), o hexágono representa um agente responsável por um goal, e os círculos nos pontos onde várias setas se encontram indicam refinamento AND — ou seja, o goal pai só é satisfeito quando todos os sub-goals filhos forem satisfeitos.

2.5. Specification by Example (Example Mapping)

Essa metodologia desenvolve as *features* eleitas na etapa de Decisão definindo suas especificações em alto nível de abstração, porém de forma que sejam testáveis. Essa abordagem é utilizada no *Behaviour Driven Development* (BDD), estratégia de desenvolvimento ágil orientada a testes na qual exemplos são recursos de especificação.

Comportamentos desejados do sistema são representados em forma de **regras** e **exemplos**. Toda *feature*, atrelada a uma **história**, deve ser definida por regras que garantem que ela contribui para um ou mais objetivos do projeto. Também devem ser trazidos possíveis cenários que demonstram como essa regra é obedecida pelo sistema e o objetivo é alcançado. Todo cenário delimita um contexto em que o sistema toma uma ação e há um resultado esperado. Se não há conflito entre regras e o resultado esperado é sempre atingido, os testes da especificação são satisfeitos.

O mapa é composto por 4 tipos de bloco: história, regra, exemplo e perguntas (questionamentos ainda não esclarecidos). Primeiro é discutida uma história que ilustra um *goal*, para a qual são definidas regras (dispostas lado a lado). Cada regra pode ser demonstrada em exemplos (abaixo dela). Todos os questionamentos relevantes para as histórias, porém ainda não esclarecidos, devem estar no mapa.

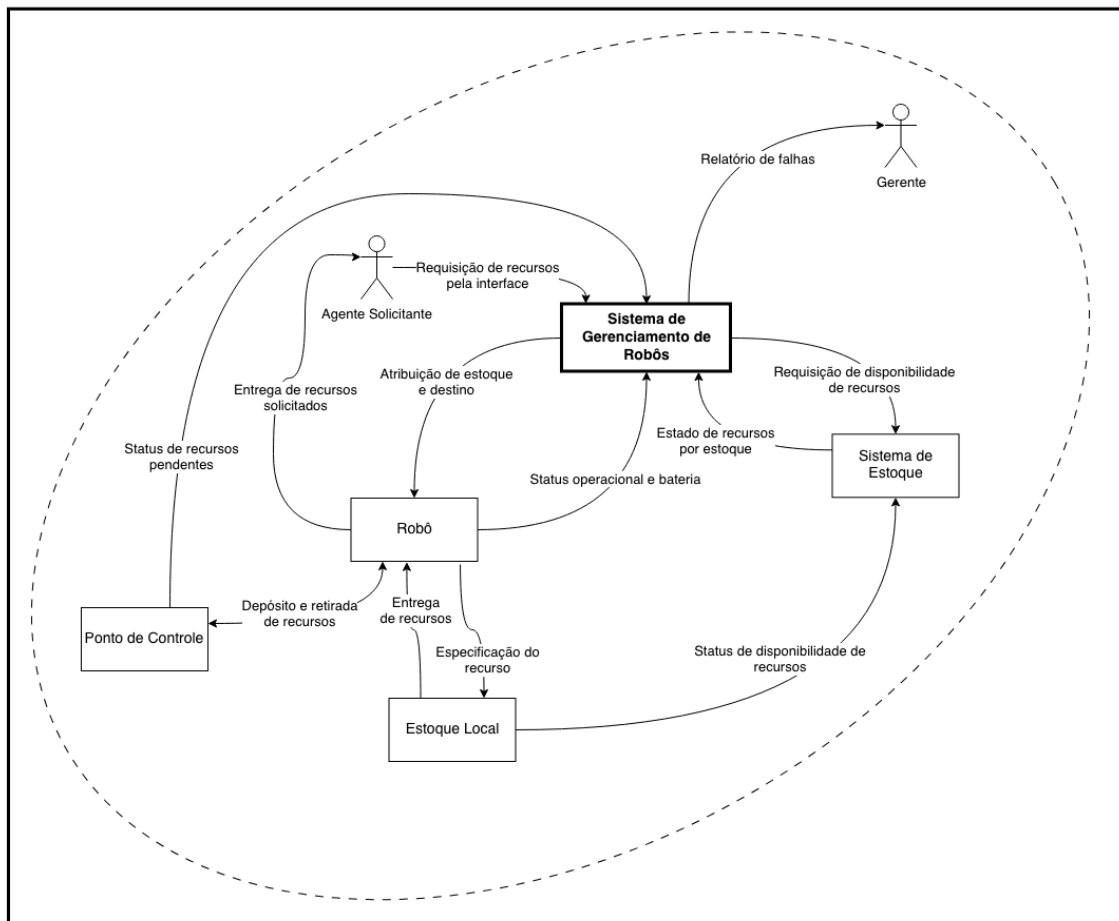
2.6. Impact Mapping

A técnica busca traçar meios para atingir os objetivos do projeto. Nela é escolhido um *goal* para ser visado; quais **agentes** podem contribuir para que ele seja alcançado; que **impactos** esses agentes podem gerar visando esse objetivo; e quais **artefatos/entregáveis** permitiriam a realização desses impactos.

Esse processo combina os conceitos trabalhados nas fases de Iniciação, Investigação e Decisão em um único mapa, de forma que os *stakeholders* considerados desempenham o papel de agentes, os impactos representam as mudanças desejadas no Mundo-como-ele-é e os entregáveis correspondem às *features* que serão eleitas para o refinamento nas fases seguintes do processo de Engenharia de Requisitos.

3. Fase 1: Iniciação

3.1. Diagrama de Contexto



A imagem acima mostra como modelamos um Diagrama de Contexto no contexto do nosso problema. Um dos primeiros desafios encontrados foi o seguinte: como funcionaria a interação entre agente solicitante e sistema? Como estamos falando em automatizar processos e permitir pedidos remotos, uma das primeiras opções pensadas foi permitir pedidos via celular por um aplicativo que realiza a interface com o sistema. No entanto, após certa reflexão, notamos que o celular é um aparelho normalmente pouco higienizado e não seria seguro manuseá-lo para fazer pedidos remotos importantes, durante, por exemplo, um procedimento cirúrgico. Além disso, estaria suscetível a eventuais problemas de conexão, de bateria e de mau funcionamento ocasional.

Com tudo isso em mente, o nosso diagrama de contexto foi elaborado em volta da ideia de que teríamos painéis eletrônicos simples (que serviriam como uma interface com o sistema) acoplados em paredes ou suportes em todas as salas de hospital que precisassem desse tipo de serviço de “delivery” de recursos. Nesse painel – que poderia ser manuseado apenas por enfermeiros e pessoal autorizado com a posse de um crachá de identificação, visto que o uso por pacientes poderia acarretar problemas de segurança e de logística – o agente solicitante poderia escolher o material que precisa e a sala em que ele deve ser entregue, não necessitando que seja a mesma em que o pedido foi feito.

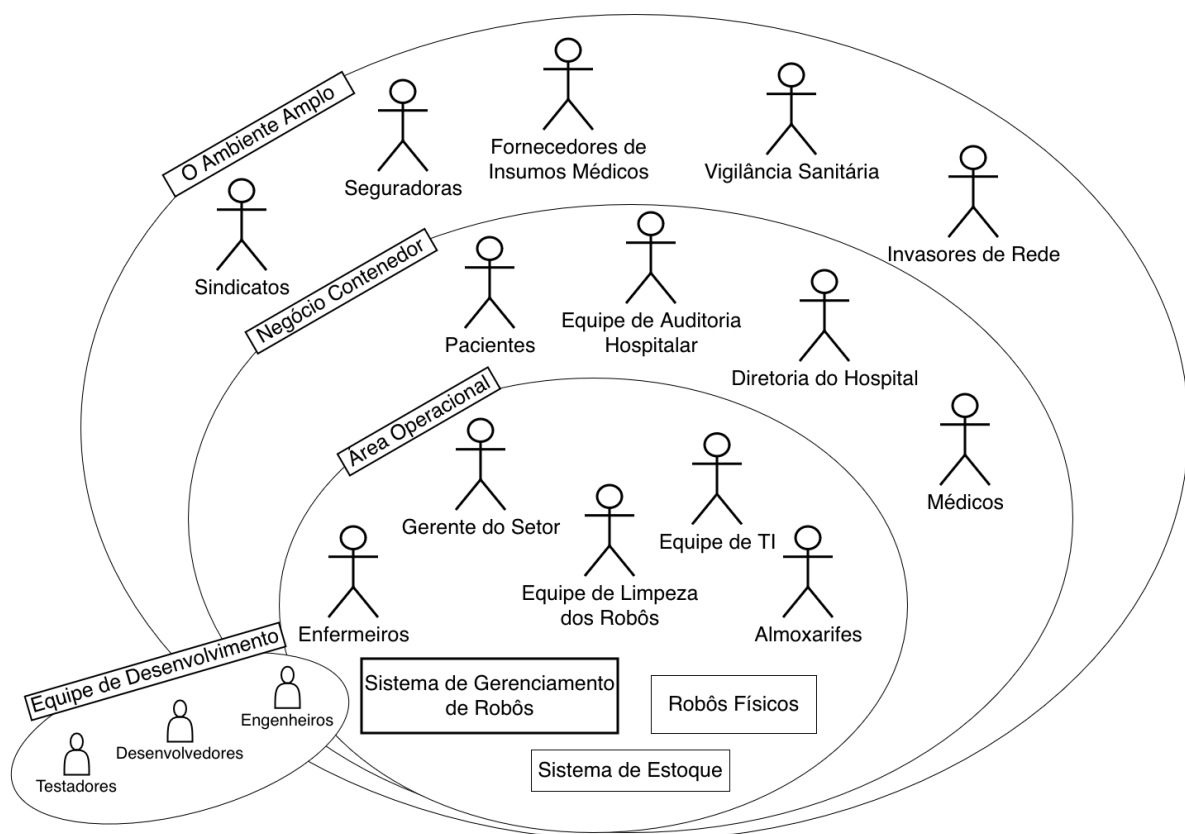
Após selecionar as exigências, o sistema realiza seus cálculos e processos para estimar o tempo de serviço e todos os outros detalhes internos à máquina. O sistema também interage com o software nativo de estoque de hospitais, podendo averiguar se há disponibilidade de recursos e em quais compartimentos hospitalares (considerando que haja mais de um) são encontrados. Em seguida,

o sistema atribui o respectivo estoque e destino a um robô, que possui o objetivo de realizar a coleta do material no local designado e entregá-lo ao agente solicitante na sala indicada. Quando for o momento da coleta, o robô emitirá uma mensagem deixando claro qual material precisa coletar, visto que outros robôs podem estar no mesmo local e o controle logístico e sanitário precisa ser rigoroso.

Se houver algum problema envolvendo bateria durante locomoção, o robô deposita o recurso em um ponto de controle e se dirige a uma estação de recarga (entidade passiva, logo não esclarecida no diagrama). O sistema, então, com o status de cada ponto de controle, se encarrega de direcionar um outro robô para retirar a entrega e completar o serviço.

Caso haja alguma falha, o sistema emitirá um alerta e enviará um relatório para o gerente encarregado. Frotas maiores de robôs simultâneos e várias viagens para realizar uma entrega são consideradas lógicas internas à máquina e, portanto, são omitidas nesta etapa.

3.2. Stakeholder Onion Diagram



A imagem acima ilustra o mapeamento das partes interessadas no ecossistema do nosso projeto. Assim como a definição das interfaces do sistema exigiu uma análise detalhada sobre higiene, controle de acesso e logística, a identificação dos *stakeholders* demandou uma visão ampla sobre quem afeta e quem é afetado pela automação, tentando ir além da interação direta entre o usuário e a máquina.

No centro do modelo, estabelecemos o produto idealizado: o Sistema de Gerenciamento de Robôs, que atuará como o orquestrador de toda a inteligência de roteamento e telemetria.

Na Área Operacional, posicionamos as entidades que lidam diretamente com o fluxo de trabalho do sistema. Estão presentes os Enfermeiros, que operam os painéis eletrônicos nas alas, e o Gerente do Setor, que recebe os alertas e relatórios de falha (como nos casos de descarregamento de bateria). Incluímos também os Almojarifes, que interagem com o sistema de forma indireta no momento em que o robô chega ao estoque para a coleta. Dadas as nossas premissas de que o ambiente hospitalar exige rigor sanitário e de que os painéis físicos precisam de manutenção, adicionamos a Equipe de Limpeza dos Robôs e a Equipe de TI como suporte operacional indispensável. As tecnologias (interfaces) com as quais a máquina conversa – os Robôs Físicos e o Sistema de Estoque – também figuram nesta camada.

Indo para o Negócio Contenedor, mapeamos aqueles que absorvem os benefícios e os impactos indiretos da nossa solução. A decisão de restringir o uso dos painéis eletrônicos apenas a funcionários autorizados com crachá colocou os pacientes fora da operação direta. No entanto, eles, juntamente com os médicos, são os beneficiários funcionais primários, pois usufruem da agilidade na entrega de insumos e equipamentos estéreis durante os procedimentos. A Diretoria do Hospital e a Equipe de Auditoria Hospitalar compõem esta camada como os patrocinadores do projeto e os interessados na rastreabilidade e na eficiência logística geradas.

No Ambiente Mais Amplo, consideramos as forças e restrições externas. A necessidade de um controle logístico e sanitário, mencionada anteriormente, é ditada pelas normas da Vigilância Sanitária. Fatores socioeconômicos e logísticos de grande escala são representados pelos Sindicatos, pelas Seguradoras e pelos Fornecedores de Insumos Médicos.

Ainda nesta camada externa, um ponto de relevante importância arquitetural é a presença de Invasores de Rede. Como o projeto envolve pedidos remotos de recursos médicos essenciais e frotas de robôs se comunicando continuamente, a segurança da informação é vista como um tópico a ser levado em conta. A inclusão desse ator hostil justifica a exigência de uma infraestrutura mais sólida, exigindo a implementação de protocolos de rede robustos, como a adoção de mTLS para garantir a autenticação mútua e sigilo perfeito entre os terminais nas paredes, os robôs e o servidor, além do uso de algoritmos consolidados como RSA e SHA-256 para a assinatura digital dos relatórios de falha e comandos de missão. Isso se torna ainda mais evidente quando consideramos que estamos lidando com deslocamento de recursos que podem afetar tanto positivamente quanto negativamente – caso ocorra entrega equivocada de insumo – a saúde de um paciente.

Por fim, cruzando todas essas camadas, temos a Equipe de Desenvolvimento (Engenheiros, Desenvolvedores e Testadores). É este grupo que carrega a responsabilidade de traduzir todas as restrições sanitárias, de segurança e de negócio em lógicas internas do sistema – como o gerenciamento das viagens múltiplas, o paralelismo das frotas e o cálculo constante de tempo de serviço.

4. Fase 2: Investigação

4.1. *Domain Scenarios*

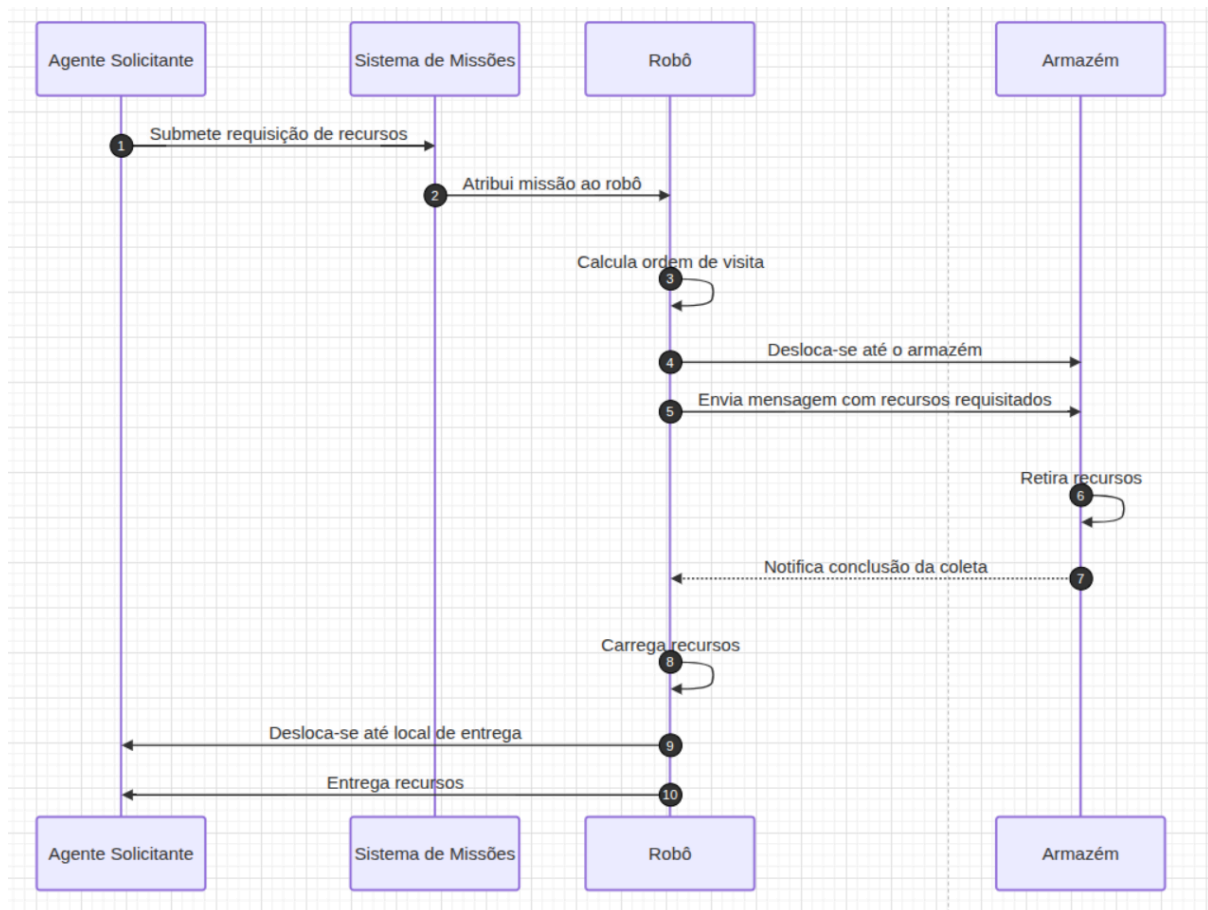
Os *domain scenarios* apresentados a seguir seguem a lógica de simular situações que podem ocorrer no contexto do sistema, buscando também ilustrar variações do cenário normal que podem ocorrer e que ficaram implícitos na descrição em linguagem natural do sistema robótico. Além disso, os cenários de falha, isto é, quando a missão não é completada, também foi explorada no cenário 5.

Cenário 1 - Robô coleta e entrega recursos de um único armazém (cenário normal)

Contexto: Um agente solicitante requisitou recursos disponíveis em um único armazém, em quantidade que cabe em uma única viagem do robô.

Resultado: O robô entrega todos os recursos solicitados ao agente solicitante no local especificado.

1. O agente solicitante submete uma requisição de recursos, com a localização de entrega, ao sistema de missões.
2. O sistema de missões atribui a missão a um robô.
3. O robô calcula a ordem de visita aos armazéns com base no tempo de espera estimado e no caminho a percorrer.
4. O robô se desloca até o armazém.
5. O robô envia uma mensagem ao armazém especificando os recursos requisitados.
6. O armazém retira os recursos requisitados e notifica o robô.
7. O robô carrega os recursos e se desloca até o local de entrega especificado.
8. O robô entrega os recursos ao agente solicitante.



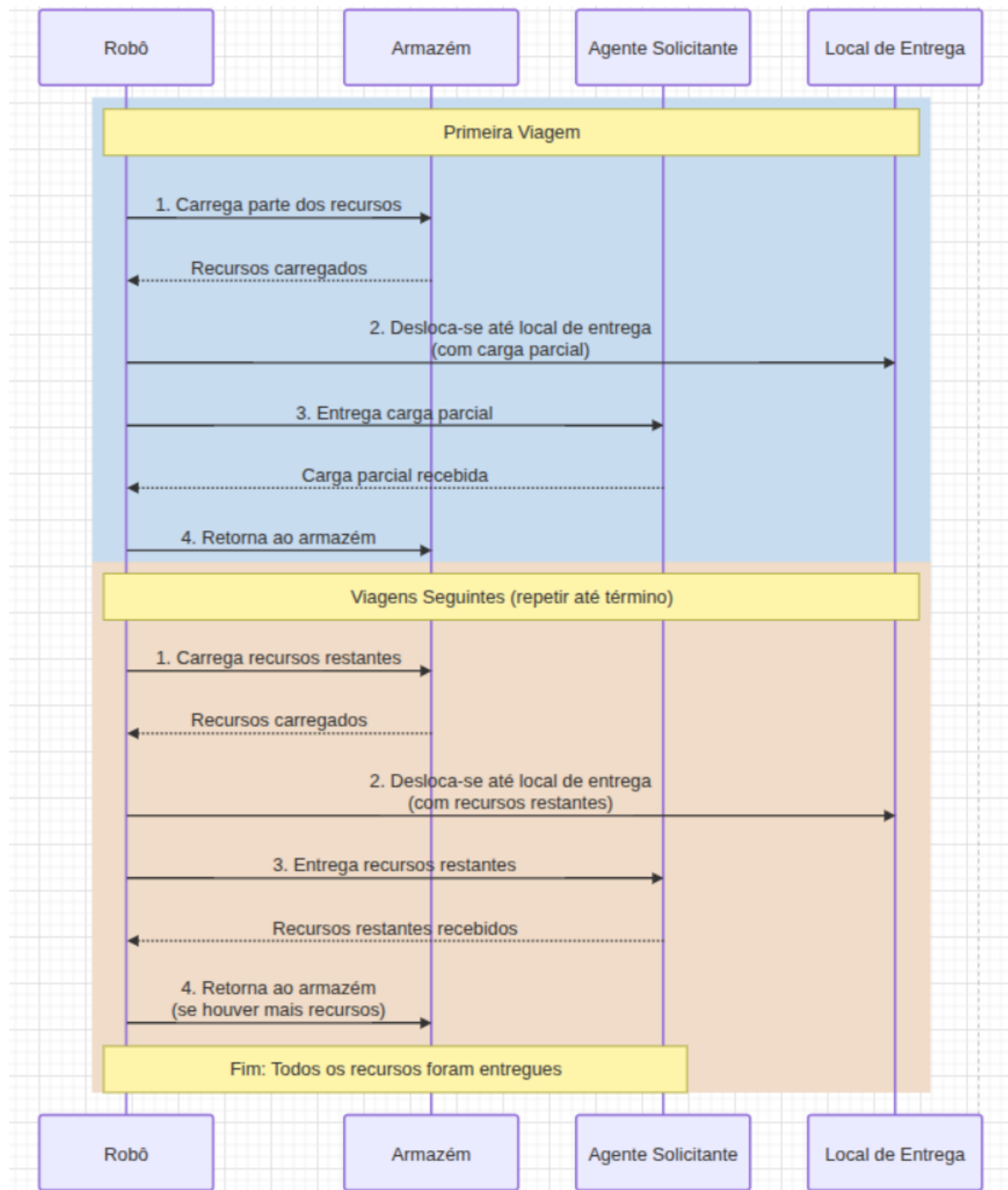
Cenário 2 - Robô entrega recursos em múltiplas viagens (variante da fase de entrega)

Contexto: O robô retirou do armazém uma quantidade de recursos maior do que sua capacidade de transporte em uma única viagem.

Resultado: Todos os recursos retirados são entregues ao agente solicitante, após múltiplas viagens.

1. O robô carrega uma parte dos recursos retirados.
2. O robô se desloca até o local de entrega especificado com a carga parcial.
3. O robô entrega a carga parcial ao agente solicitante.
4. O robô retorna ao armazém para buscar os recursos restantes.
5. O robô carrega os recursos restantes.
6. O robô se desloca novamente até o local de entrega com os recursos restantes.
7. O robô entrega os recursos restantes ao agente solicitante.

Nota: o passo 1–7 se repete tantas vezes quantas forem necessárias até que todos os recursos coletados tenham sido entregues.



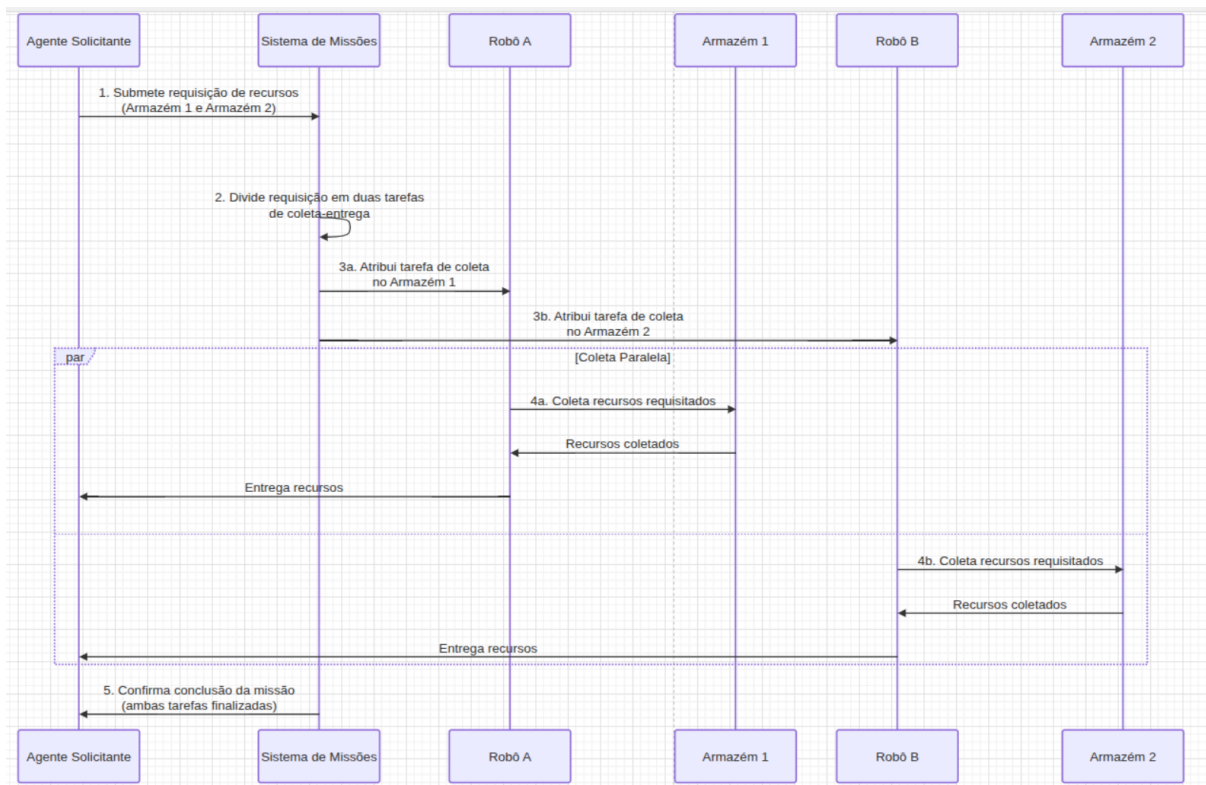
Cenário 3 - Múltiplos robôs coletam recursos de armazéns diferentes em paralelo (variante)

Contexto: O agente solicitante requisitou recursos disponíveis em armazéns diferentes.

Resultado: Todos os recursos requisitados, vindos de armazéns distintos, são entregues ao agente solicitante.

1. O agente solicitante submete uma requisição de recursos provenientes de dois armazéns distintos ao sistema de missões.
2. O sistema de missões divide a requisição em duas tarefas de coleta-entrega.
3. O sistema de missões atribui a primeira tarefa ao Robô A e a segunda tarefa ao Robô B.
4. O Robô A coleta seus recursos requisitados no primeiro armazém e os entrega ao agente solicitante.
5. Em paralelo, o Robô B coleta seus recursos requisitados no segundo armazém e os entrega ao agente solicitante.
6. O sistema de missões confirma a conclusão da missão ao agente solicitante quando ambas as tarefas são finalizadas.

Nota: Os passos 4 e 5 ocorrem em paralelo e cada um pode, por si só, expandir-se conforme o Cenário 1 ou 2.

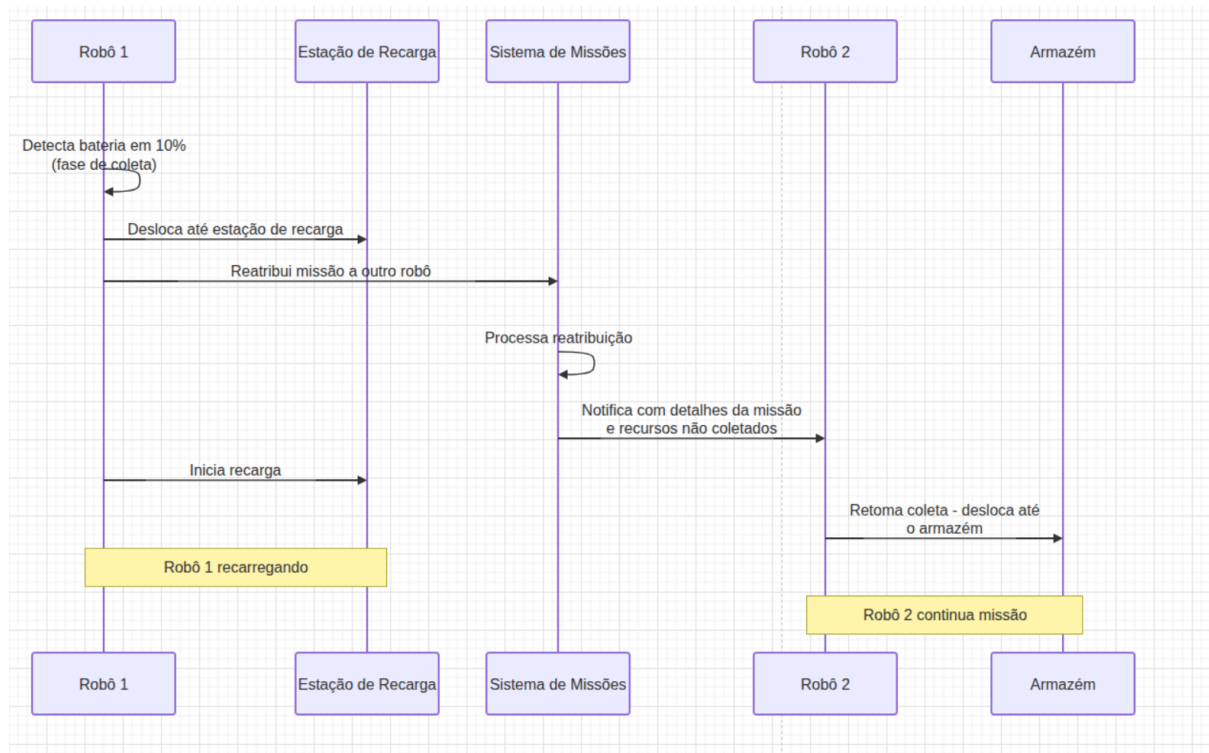


Cenário 4 - Bateria baixa durante a fase de coleta (exceção)

Contexto: Um robô está na fase de coleta de uma missão quando sua bateria atinge 10%.

Resultado: A missão é reatribuída a outro robô e o robô original está recarregando.

1. O robô detecta que sua bateria atingiu 10% durante a fase de coleta.
2. O robô se desloca até a estação de recarga.
3. O robô reatribui a missão a outro robô por meio do sistema de missões.
4. O sistema de missões notifica o novo robô com os detalhes da missão e dos recursos ainda não coletados.
5. O novo robô retoma a fase de coleta, deslocando-se até o armazém.



Cenário 5 - Bateria baixa durante a fase de entrega (exceção, com variante e exceção interna)

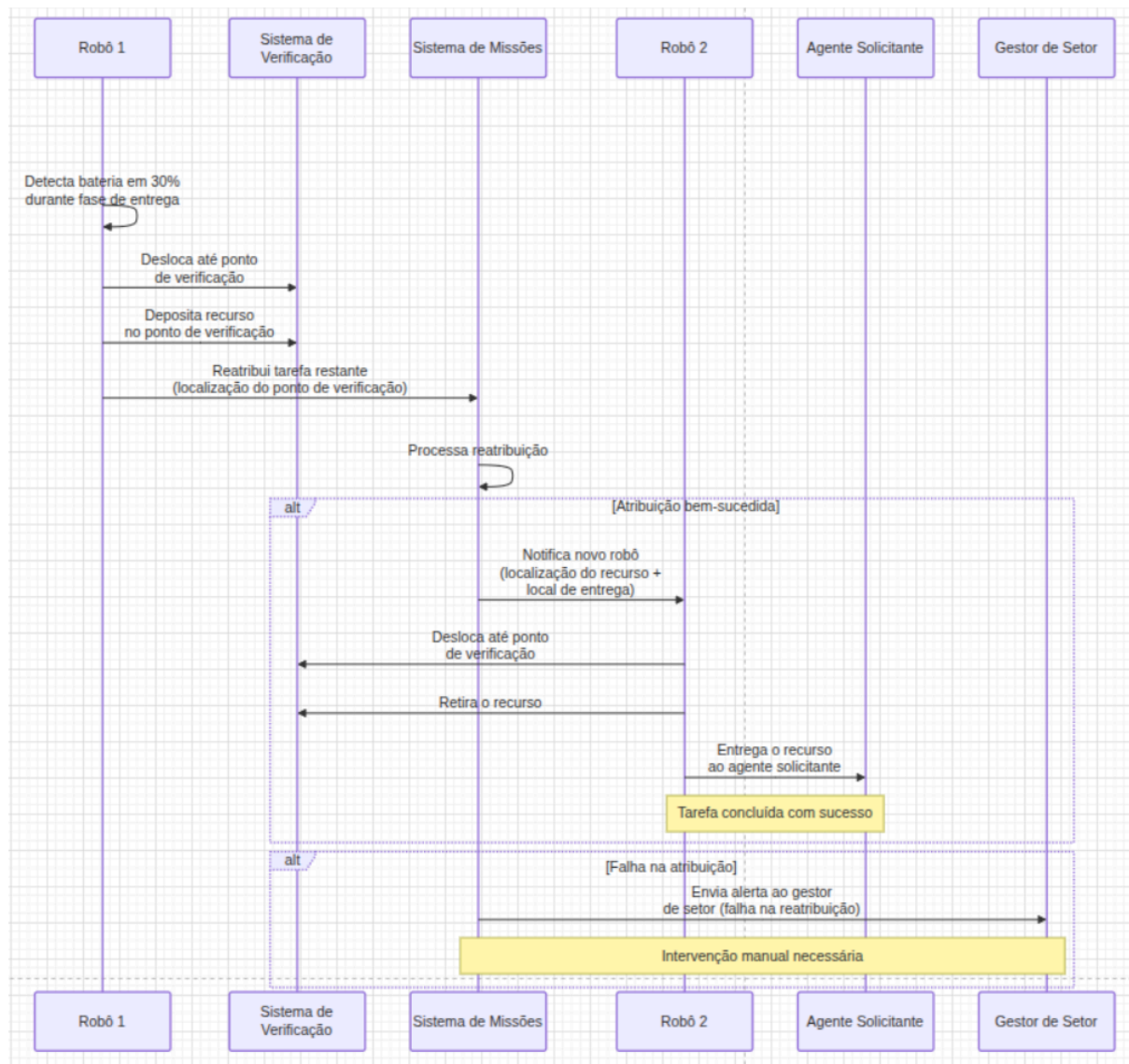
Contexto: Um robô está transportando recursos na fase de entrega quando sua bateria atinge 30%.

Resultado: O recurso é entregue por outro robô a partir do ponto de verificação, ou, em caso de falha, um alerta é enviado ao gestor de setor.

1. O robô detecta que sua bateria atingiu 30% durante a fase de entrega.
2. O robô se desloca até um ponto de verificação.
3. O robô deposita o recurso no ponto de verificação.
4. O robô reatribui a tarefa restante a outro robô por meio do sistema de missões, informando a localização do ponto de verificação.
5. O sistema de missões notifica o novo robô com a localização do recurso e o local de entrega.
6. O novo robô se desloca até o ponto de verificação e retira o recurso.
7. O novo robô entrega o recurso ao agente solicitante no local especificado.

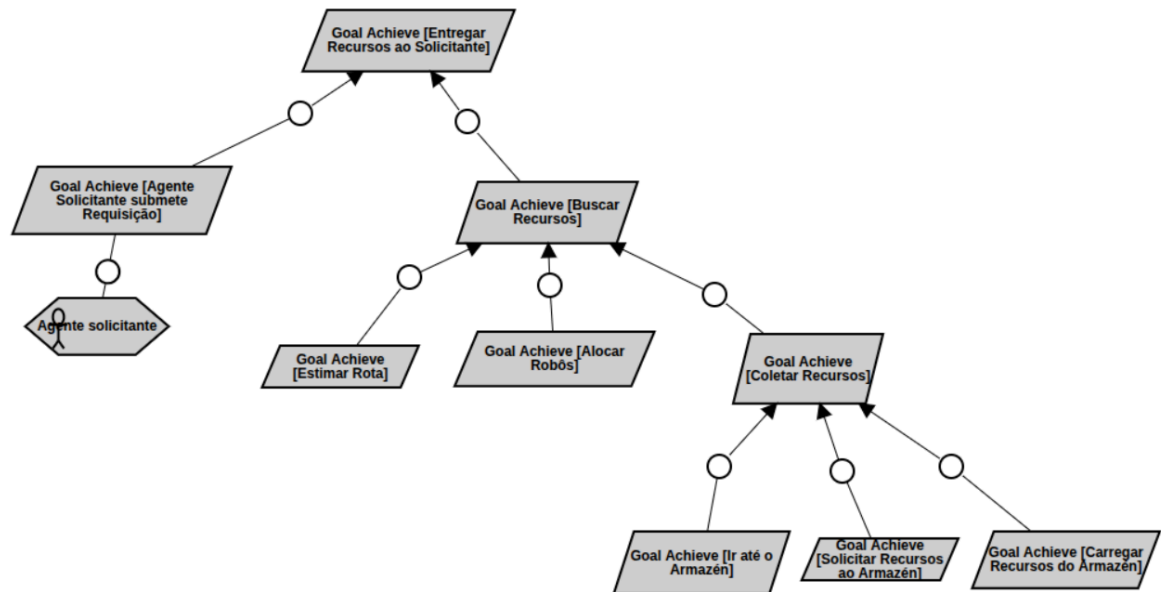
Exceção ao passo 3 (possíveis variações): Se o robô falhar ao depositar o recurso no ponto de verificação:

- O robô aciona um alerta.
- O robô envia um relatório do incidente ao gestor de setor.



4.2. Goal Model (v1)

Com o cenário esboçado pelos *Domain Scenarios*, é possível delinear uma primeira versão do Goal Model. Focando apenas nos aspectos mais essenciais e alto-nível, podemos decompor o objetivo principal em sub objetivos. No entanto, essa primeira versão ainda não inclui cenários de adaptação e correção de falhas, que serão explorados nas próximas seções.



4.2.1 Goal raiz - Entregar Recursos ao Solicitante

Corresponde à frase de abertura da missão: o robô deve coletar os recursos requisitados e entregá-los ao agente solicitante em um local especificado. É refinado (AND) em dois sub-goals:

- **Agente Solicitante submete Requisição:** atribuído ao "Agente solicitante". Representa o gatilho "When required" do texto: a missão só começa quando o pedido é formalizado.
- **Entregar Recursos:** cobre todo o processo logístico de coleta e entrega.

4.2.2 Buscar Recursos

Refinado em três sub-goals (AND), que cobrem o planejamento e a execução:

- **Estimar Rota:** reflete "The order to be followed is defined by the estimation of waiting time summed up with the path to be run by the robot".
- **Alocar Robôs:** reflete "multiple robots can be assigned to parallel collect-deliver tasks": a decisão de quantos/quais robôs participarão da missão.
- **Coletar Recursos:** a fase de coleta, detalhada abaixo.

4.2.3 Coletar Recursos

Refinado em três sub-goals operacionais, que seguem exatamente a sequência narrada no texto:

- **Ir até o Armazém:** "the robot must go to the storage places where the resources can be found".
- **Solicitar Recursos ao Armazém:** "the robot sends a message to the storage with the precise specification of the requested resources".
- **Carregar Recursos do Armazém:** "...and waits until the resources have been retrieved" (carregamento físico no robô).

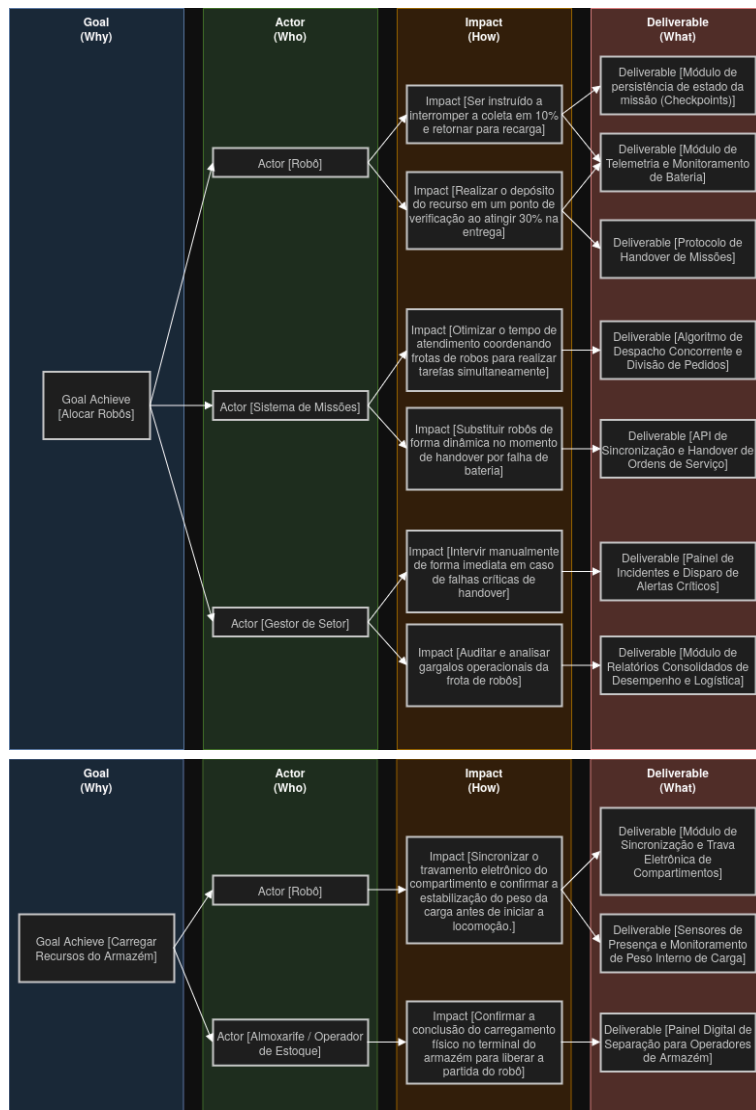
5. Fase 3: Decisão

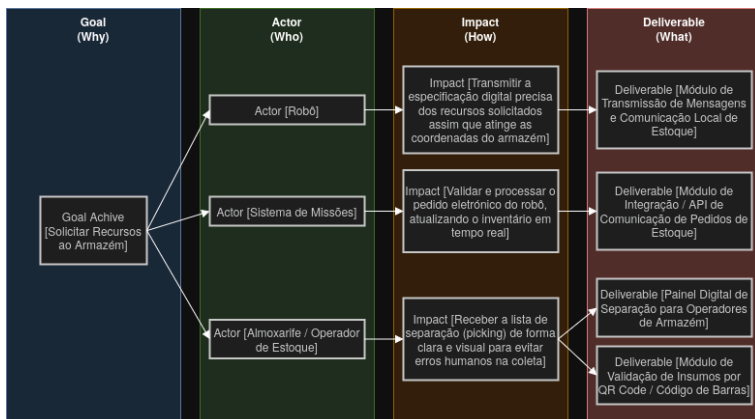
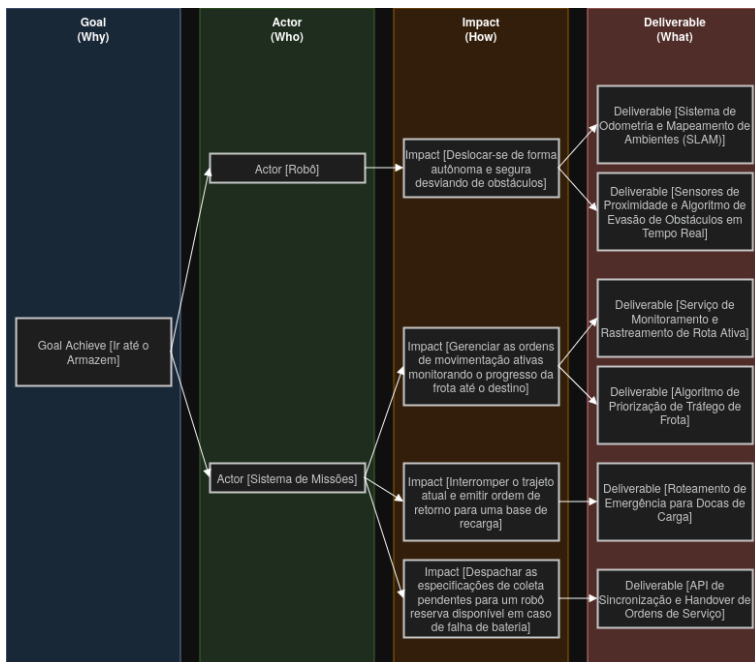
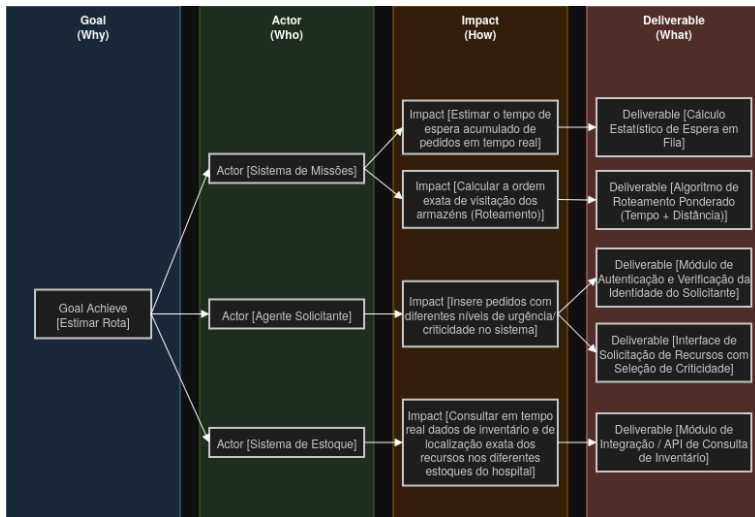
5.1. Impact Maps

Os *impact maps* desenvolvidos trazem um grau mais concreto de visualização do projeto como um todo. Os agentes considerados são: Sistema de Missões, o Robô de entrega, o Gestor de Setor, Almoxarife e Agente Solicitante. Sobre esses agentes são desenvolvidos impactos e entregáveis relacionados a diversas interfaces entre agentes, desde comunicação entre aplicações de diferentes setores a implementações de algoritmos internos do Sistema de Missões.

Os *goals* destrinchados foram:

- Goal Achieve [Alocar Robôs];
- Goal Achieve [Carregar Recursos do Armazém];
- Goal Achieve [Estimar Rota];
- Goal Achieve [Ir até o Armazém];
- Goal Achieve [Solicitar Recursos do Armazém].



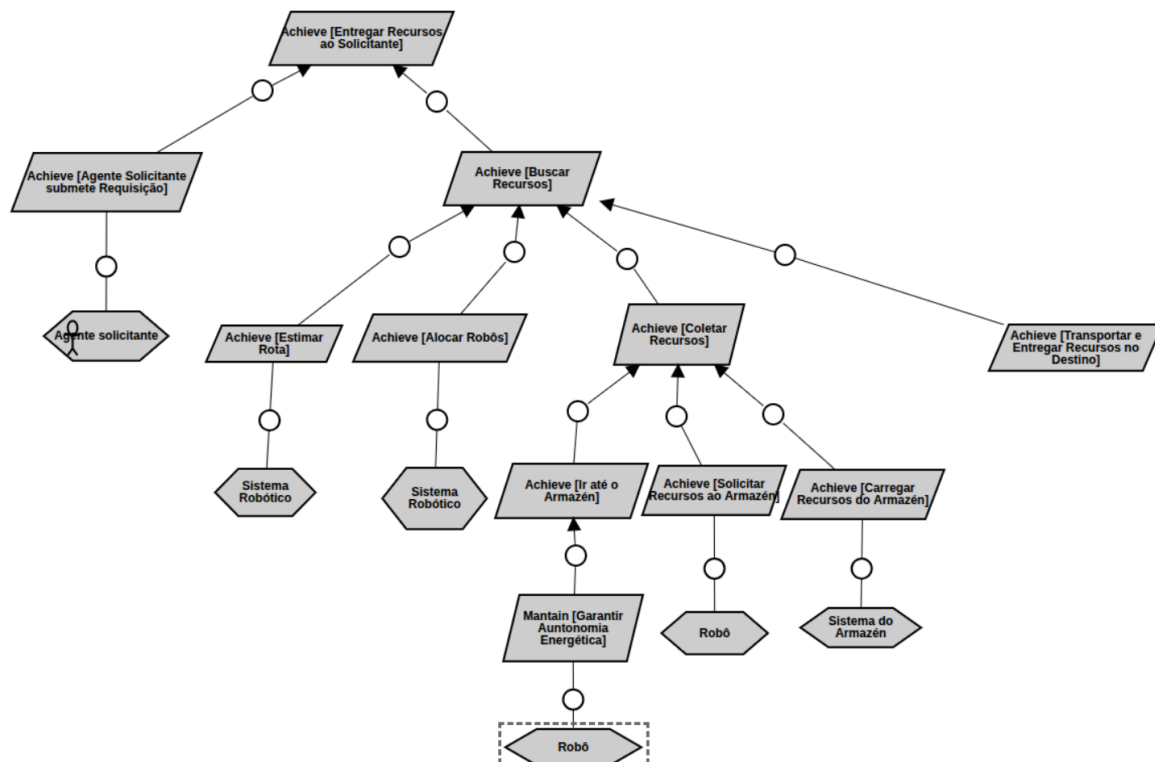


5.2. Goal Model (v2)

Com base nos *impact maps*, é possível atribuir os agentes responsáveis por cada *Goal*. Desse modo, identificamos os 4 principais agentes responsáveis pela execução e conclusão dos goals:

- Agente Solicitante (Environment Agent)
- Robô
- Sistema Robótico
- Sistema do Armazém

Além disso, podemos também estabelecer goals de manutenção, como o sub-goal “*Garantir Autonomia Energética*”. Pois o impact map também permite visualizar as condições que devem ser mantidas para que os goals subsequentes sejam alcançados.

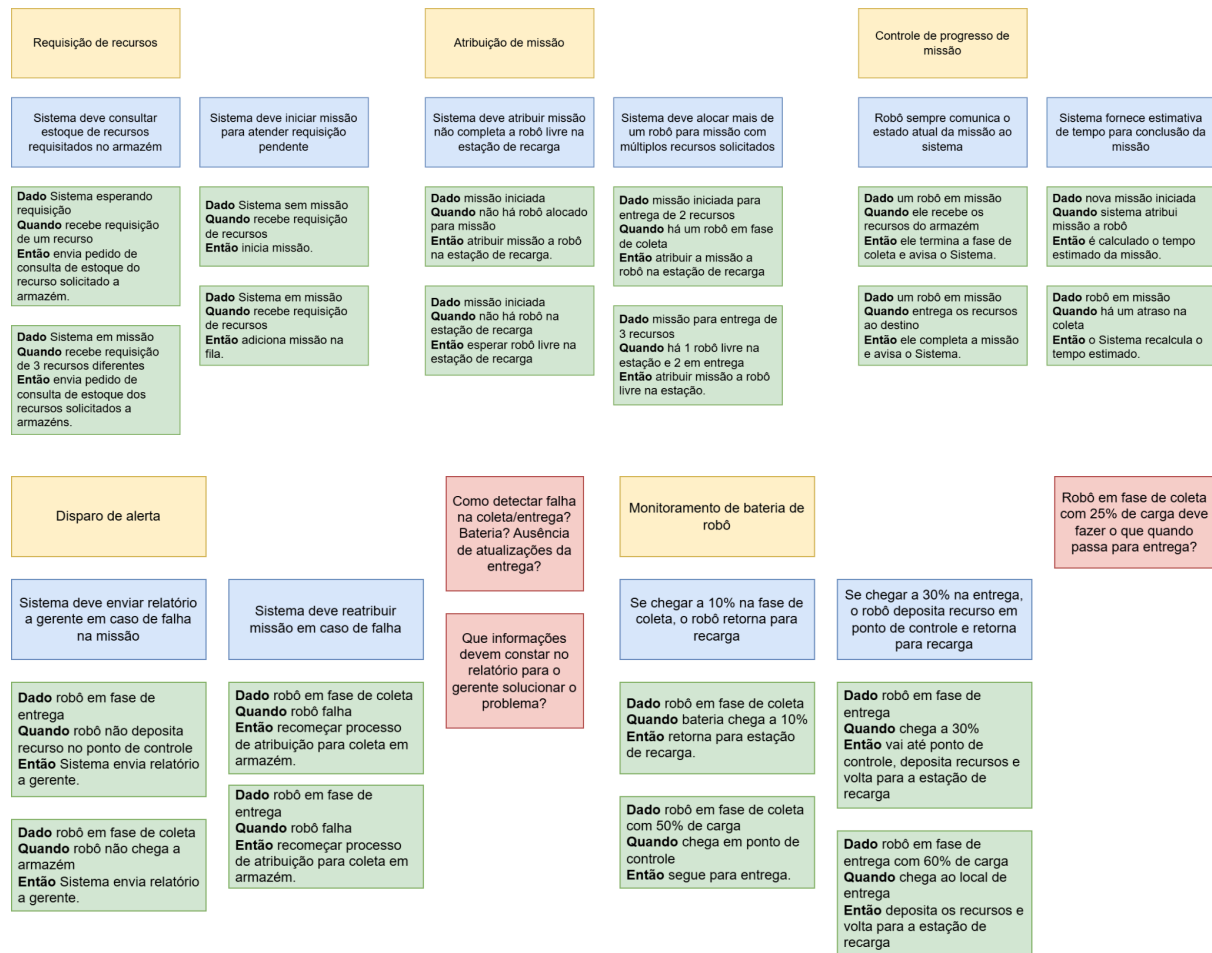


6. Fase 4: Formulação

6.1. Example Maps

Os mapas a seguir apresentam as especificações para as seguintes *features*:

- Requisição de recursos, determinando o que o Sistema deve fazer quando chega uma nova requisição;
- Atribuição de missão, indicando o comportamento desejado do sistema na alocação de robôs para completar uma missão;
- Controle de progresso de missão, definindo como os agentes devem lidar com o progresso da missão;
- Monitoramento da bateria de robô, incluindo gatilhos e ações que o robô deve tomar a depender do estado da bateria;
- Disparo de alerta, para definir a resposta adequada dos agentes em casos de falha.



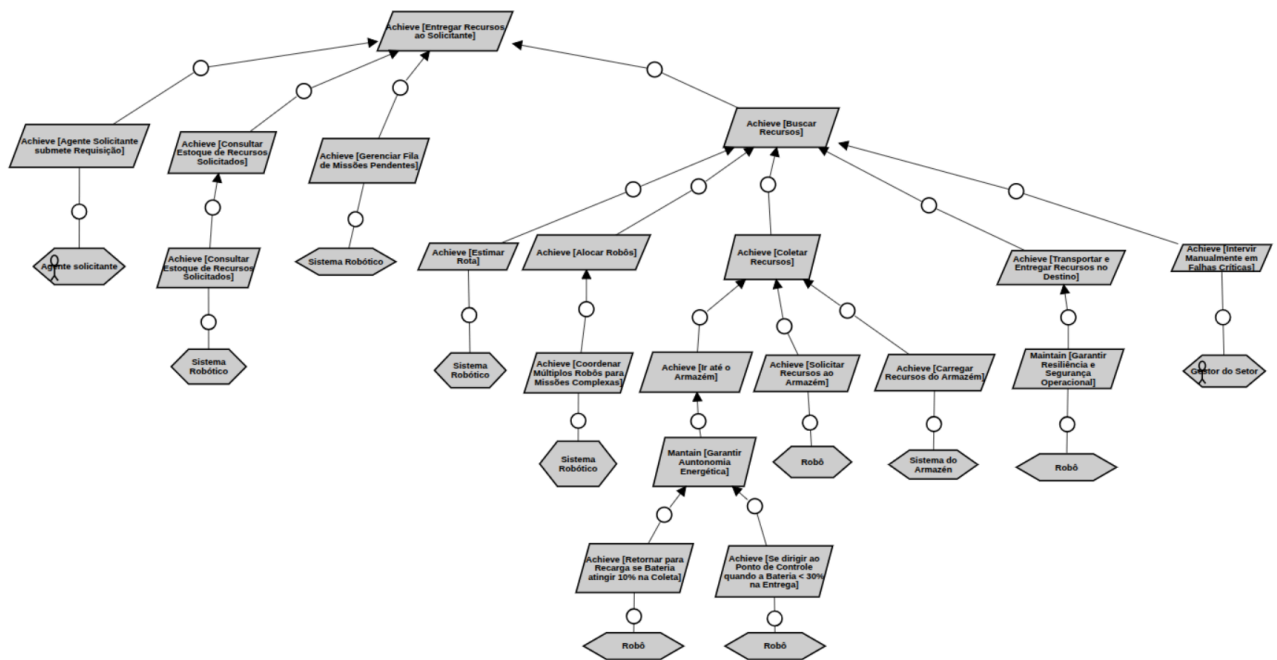
6.2. Goal Model (vFinal)

O uso dos *Example Maps* traz uma camada de cenários concretos (baseados em regras de negócio e exemplos práticos de Dado/Quando/Então) que expõe lacunas importantes no *Goal Model* que estiveram omissas ou abstratas nas versões anteriores.

Desse modo, o goal principal pode ser refinado em mais dois *sub-goals*: Consultar Estoques de Recursos Solicitados e Gerenciar fila de Missões Pendentes. Pois os *Example Maps* ilustram quanto à necessidade de verificar a localização e disponibilidade dos recursos e o gerenciamento de vários robôs executando a missão paralelamente.

Além disso, foi possível refinar objetivos voltados para resiliência e robustez do sistema, como os *goals* de tratamento de casos onde a bateria do robô está baixa e intervenção humana no caso de falhas do robô.

A visão mais concreta dos *Example Maps* e dos *Impact Maps* permitiram visualizar a necessidade de *goals* de manutenção (*Maintain*) de condições de runtime necessárias para a conclusão da missão.



Referências

- [1] LETIER, Emmanuel. **Requirements Engineering: A Concise Guide to Modern Ideas and Practices**. [S. l.], 1 fev. 2026. Disponível em: <http://www0.cs.ucl.ac.uk/staff/E.Letier/book/>. Acesso em: 26 jun. 2026.
- [2] WYNNE, Matt. **Introducing Example Mapping**. [S. l.], 8 dez. 2015. Disponível em: <https://cucumber.io/blog/bdd/example-mapping-introduction/>. Acesso em: 26 jun. 2026.